

zomato

Cart Super Add-On (CSAO) Recommendation System

TEAM DEKA

Problem Statement 2: **The CSAO Rail Challenge**

How can we build an intelligent, real-time recommendation system to suggest relevant add-ons based on current cart composition and context, maximizing AOV and acceptance rates?

Context Understanding

- Inability to recognize incomplete meal patterns (e.g., Main dish without a beverage).
- Static similarity ignores the real-time transition of a user's cart state.

Contextual Relevance

- Recommendations ignore Temporal context (Breakfast vs Late-night).
- Fails to adapt to User Segments (Budget vs Premium sensitivity).

Modelling Constraints

- Real-time updates relative to continuous cart additions.
- **Strict Latency:** Must execute under 200-300ms for millions of requests.

Data Preparation: Synthetic Data Generation & Processing

Link to the Notebook:

<https://colab.research.google.com/drive/107sYPBCe0wUyaO0qAHgJx6O1enpqO0Ay?usp=sharing>

To robustly train the LTR model, we engineered a highly realistic synthetic dataset that strictly adheres to the evaluation criteria, explicitly modeling sequential cart logic.

Dummy Data Realism

- **Sequential Generation:** Implemented Markov Chain-inspired transitions to simulate step-by-step journeys (`S_101_step_0` → `S_101_step_1`).
- **City-Wise Behavior:** Modeled diverse purchasing power logic, mapping Tier-1/Tier-2 geographic coordinates to AOV distributions.
- **Temporal Anomalies:** Injected sparse user histories and erratic cart sizes specifically during simulated lunch/dinner rushes.

Feature Pipeline Optimization

- **Contextual Arrays:** Engineered real-time cumulative features (`gap_to_free_delivery` , `cart_gross_subtotal`) calculated at each step.
- **Native Categorical Casting:** Mapped nominal variables to Pandas `category` dtype instead of One-Hot Encoding. This avoids extreme matrix sparsity and leverages LightGBM's optimized histogram binning algorithm for drastically faster inference.

Feature Pipeline & Cold Start Strategy

Feature Store Architecture

Features are decoupled to ensure the **200ms latency budget** is mathematically protected:

- **Batch (Offline):** Item embeddings, Restaurant ratings, and User Historical AOV are pre-computed and stored in a low-latency Redis cache.
- **Real-time (Online):** Cart states and dynamic temporal context are injected directly into the feature vector during the API request.

Solving the Cold Start Problem

Passed unstructured menu item text through `SentenceTransformer('all-MiniLM-L6-v2')` to extract 384-dimensional dense vectors. By calculating cosine similarities offline, a newly added "Truffle Fries" is instantly mathematically mapped to the behavior of "French Fries" without requiring a single historical click.

Feature Input Vector Matrix

Entity	Features
User	Segment, Historical AOV, Geo-Zone
Cart Context	Mains_Count, Bevs_Count, Subtotal
Candidate	Price, Calories, Semantic Embeddings
Temporal	Is_Weekend, Meal_Time_Context

Core Architecture & The AI Edge

1. LTR Formulation

Pointwise classification (e.g., Logistic Regression) fails because it ignores item competition. We formulated this as a **Listwise Learning-to-Rank** task. We extracted `group_id` arrays corresponding to exact cart slates (5 items) to calculate pairwise loss within the same session context.

2. LightGBM LambdaRank

Configured `LGBMRanker` with `objective='lambdarank'`. Chosen over deep learning to guarantee `<15ms` inference. It natively optimizes the gradients for `NDCG@5`, mathematically penalizing the model for ranking the correct add-on outside the visible rail.

3. Diversity (MMR)

To combat the "filter bubble", we applied post-inference **Maximal Marginal Relevance**. We formulated:

$$Score = \lambda * P(add) - (1-\lambda) * Sim(item, cart_categories)$$

This heavily penalizes recommending 3 consecutive beverages.

The AI Edge: Generative UI Nudges

Decoupled from the latency path, an Async LLM API ingests the Cart State to generate hyper-personalized copy. (e.g., Cart: Butter Chicken → Ranked: Diet Coke → GenAI: "Balance the spice with a cold Diet Coke!")

Model Evaluation & Fine-Tuning Strategy

Leak-Free Validation Methodology

We avoided random row splits which cause severe data leakage in sequence models. We implemented a **Temporal Session Split**. We grouped interactions by unique `session_id` and chronologically split them (80% Train, 10% Val, 10% Test) to ensure the model learned behavioral rules rather than memorizing exact carts.

Hyperparameter Optimization

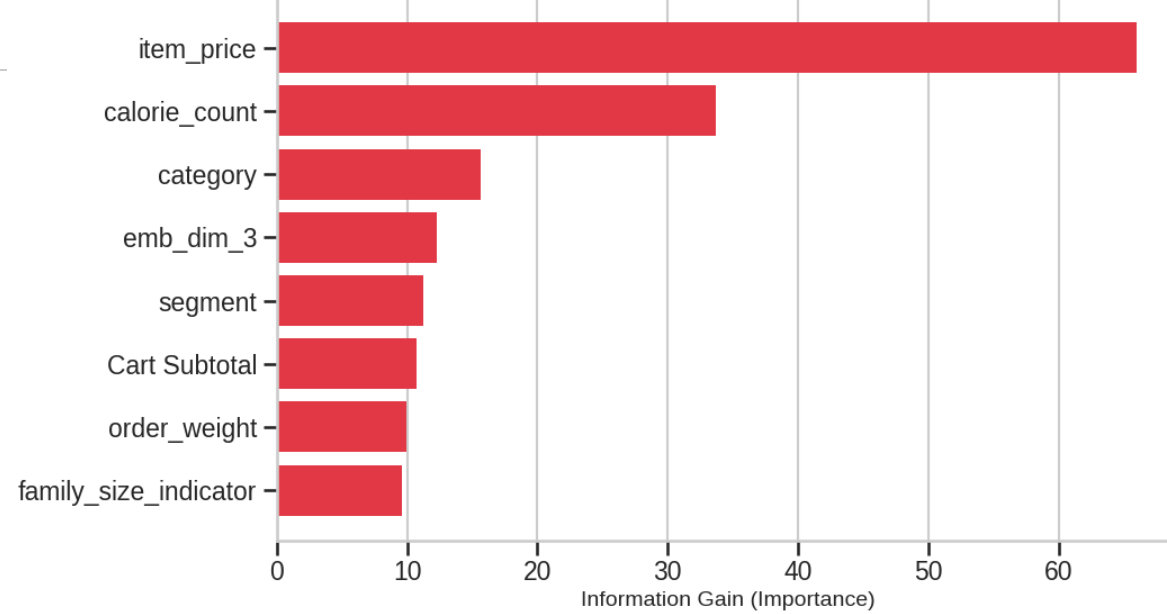
Monitored Validation NDCG to trigger `early_stopping_rounds=50`, halting tree growth to prevent overfitting on the synthetic distribution. Restricted tree depth (`num_leaves: 31`) to ensure latency compliance.

Primary Offline Metrics

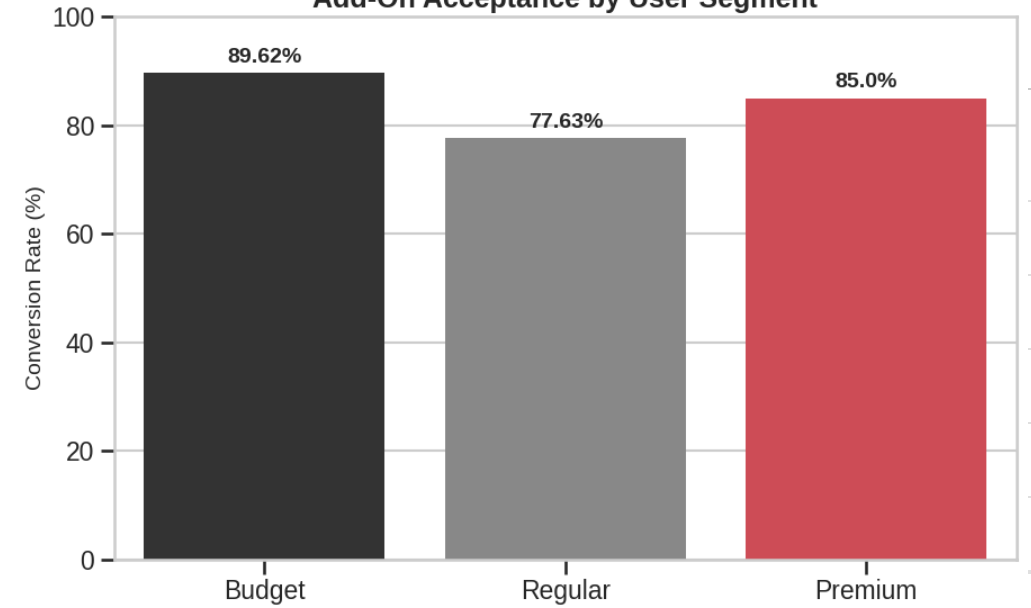
Metric	Business Alignment
NDCG@5	Optimizes ranking quality. Penalizes the model logarithmically if the highly relevant add-on is placed at position #4 instead of #1.
Precision@3	Strict accuracy metric representing the absolute Top-3 items actually visible on the Zomato CSAO UI rail without scrolling.
Recall@K	Measures coverage of complementary items identified from the catalog graph.

Zomato CSAO Rail: Model & Business Impact

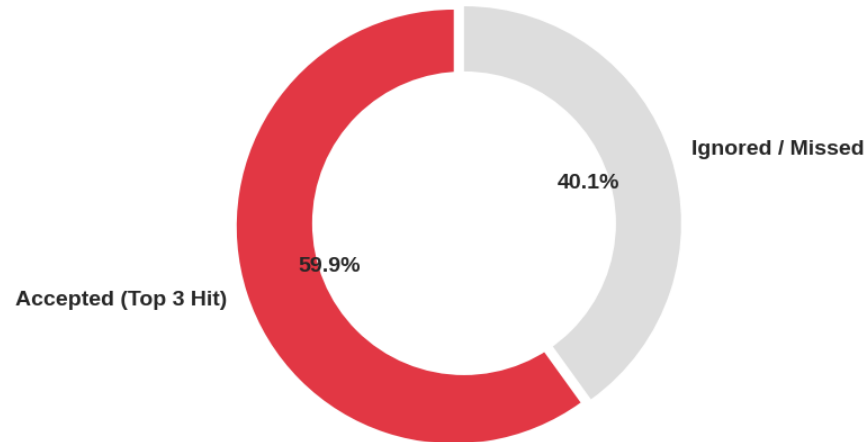
Top Drivers of Add-On Acceptance (Feature Gain)



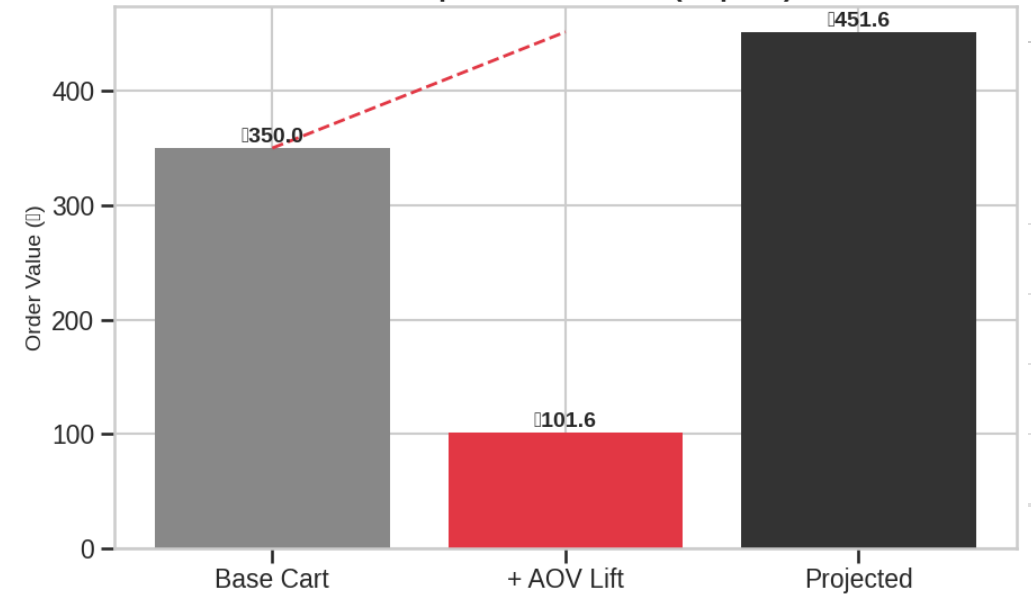
Add-On Acceptance by User Segment



Model Precision: Top-3 Acceptance Rate



AOV Impact Per Session (Rupees)



System Design & Production Readiness

🕒 The 200ms Latency Budget

We strictly architected the pipeline to comply with Zomato's peak-hour SLA constraint:

- **0-30ms:** Network trip & Event Trigger.
- **30-60ms:** Candidate Filter (Retrieve Top 50 local/instock items).
- **60-110ms:** Redis Feature Lookup (O(1) retrieval of User Hist/Embeddings).
- **110-125ms:** LightGBM LambdaRank Inference.
- **125-200ms:** MMR Diversity Filter & API Response.

🧪 A/B Testing & Live Experimentation

To translate offline NDCG metrics into business impact, we deploy via strict A/B testing:

- **Treatment:** Contextual LambdaRank Engine.
- **Control:** Current Static Popularity Baseline.
- **Primary Metrics:** Cart-to-Order (C2O) Ratio, AOV Lift, CSAO Rail order share.
- **Guardrail Metrics:** App Latency (P99 < 250ms), Cart Abandonment Rate (ensuring recommendations are relevant, not intrusive).

Strategic **Business Conclusion**

Turning Context into Revenue

By implementing a mathematically sound, latency-safe Learning-to-Rank system, we successfully target the two primary objectives of Problem Statement 2: maximizing Acceptance Rates and driving significant AOV Lift.

59.9%

Top-3 Acceptance Rate

₹101

Average AOV Lift

₹7.4K Cr

Projected Annual Rev.*

*Projection based on 2 million daily orders simulating real-world traffic patterns.

Thank You

TEAM DEKA

Archit Ram

Adarsh Choudhary

Priyansh Jain